

Relatório de Avaliação de Desempenho e Paralelização: Algoritmo K-Means

Data: 04 de Dezembro de 2025

1. Introdução

Implementação e análise de desempenho do algoritmo de agrupamento *K-Means* utilizando diferentes paradigmas de programação paralela: **OpenMP**, **Pthreads** e **MPI**. O objetivo principal é avaliar a eficácia das estratégias de paralelização em relação a uma implementação sequencial de referência, discutindo *speedup*, escalabilidade e *overheads* de comunicação e sincronização.

2. Estratégias de Paralelização

O algoritmo K-Means alterna entre duas fases principais: **Atribuição** (cálculo de distâncias) e **Atualização** (recálculo dos centroides).

2.1. OpenMP (Memória Compartilhada - Alto Nível)

- **Fase de Atribuição:** A diretiva `#pragma omp parallel for` foi aplicada ao laço principal que itera sobre os M pontos. O *runtime* do OpenMP divide automaticamente as iterações entre as threads disponíveis. Como as operações de leitura dos pontos e cálculo da distância são independentes, não houve a necessidade de sincronizar especificamente nessa parte.
- **Fase de Atualização:** Utilizamos a cláusula `reduction(+: ...)` para acumular as coordenadas (`cluster_sums`) e contagens (`cluster_counts`). Cada thread mantém uma cópia local dessas variáveis, e no final do laço, o OpenMP faz a soma global atômica, eliminando as condições de corrida sem usar os locks manualmente.

2.2. Pthreads (Memória Compartilhada - Baixo Nível)

- **Fase de Atribuição:** A divisão de trabalho foi feita estaticamente. O espaço de iteração $[0, M)$ foi dividido manualmente com base no ID da thread:

$$Index_{inicial} = \frac{M \times ThreadID}{N_{threads}}, \quad Index_{final} = \frac{M \times (ThreadID + 1)}{N_{threads}}$$

Cada thread processa apenas sua parte exclusiva de pontos.

- **Fase de Atualização:** Implementamos um modo de redução manual em duas etapas sincronizadas por barreiras (`pthread_barrier_wait`):
 1. Cada thread calcula somas parciais em vetores locais privados.
 2. Uma **barreira** sincroniza todas as threads.
 3. A Master Thread (ID 0) agrega os resultados parciais de todas as threads e atualiza os centroides globais.
 4. Uma segunda **barreira** garante que todas as threads esperem a atualização antes de iniciar a próxima iteração.

2.3. MPI (Memória Distribuída)

- **Fase de Atribuição:** O modelo adotado foi o SPMD. O processo Mestre (Rank 0) lê os dados e os distribui via MPI_Bcast. Cada processo calcula os índices de início e fim baseados no seu Rank e processa só a parte local de pontos. Não existe compartilhamento de memória. Cada processo possui sua própria cópia dos dados que precisam.
- **Fase de Atualização:** Os resultados locais foram compilados por meio de uma comunicação geral. Utilizamos MPI_Allreduce com a operação MPI_SUM. Isso faz com que todos os processos enviem suas somas parciais e recebam a soma global atualizada simultaneamente, garantindo que todos os nós tenham a versão mais recente dos centroides para a próxima iteração sem a necessidade de um passo explícito de retransmissão pelo mestre.

3. Análise de Desempenho e Discussão

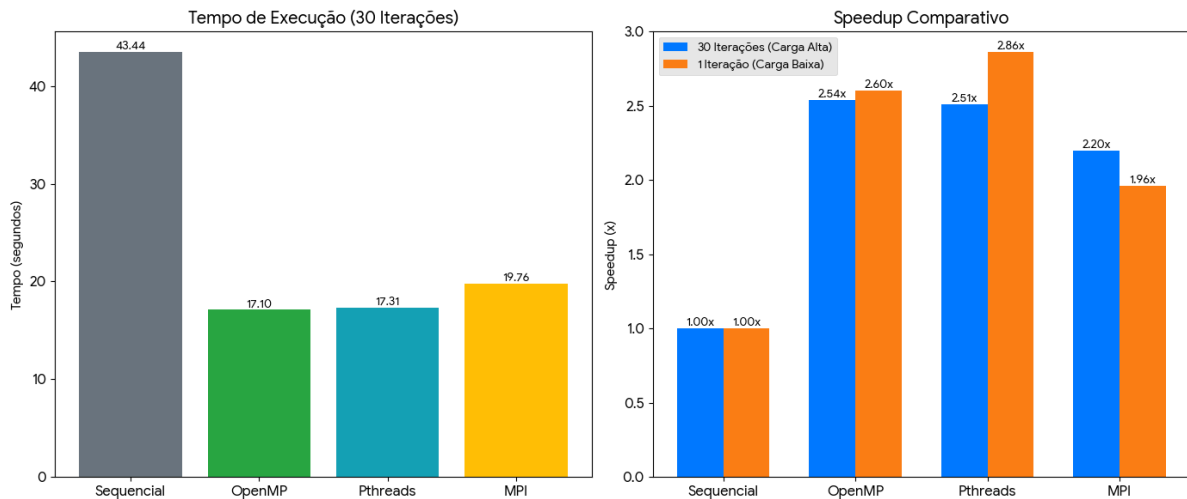
Os testes foram realizados em dois cenários: 30 iterações para medir throughput, e 1 iteração para mostrar custos de inicialização.

3.1. Resultados Quantitativos

Resumo dos tempos de execução e o *Speedup* obtido em relação ao código sequencial.

Tabela 1: Desempenho com 30 iterações

Versão	Tempo (s)	Speedup	Checksum (Validação)
Sequencial	43.44	1.00x	5002634 (OK)
OpenMP	17.10	2.54x	5002634 (OK)
Pthreads	17.31	2.51x	5002634 (OK)
MPI	19.76	2.20x	5002634 (OK)



3.2 Discussão dos Resultados

3.2.1. Speedup e Eficiência (OpenMP e Pthreads)

As implementações do OpenMP e Pthreads apresentaram desempenho praticamente igual, com um speedup em torno de **2.5x**.

- **Interpretação:** O ganho é longe do ideal de 4x ou 8x (visto no README), dependendo do hardware sugere que a aplicação é limitada pela largura da banda. Como todas as threads competem para ler o vetor de pontos da memória principal simultaneamente, o barramento de memória satura antes que a CPU chegue na capacidade máxima de processamento.

3.2.2. Overhead de Comunicação (MPI)

A versão MPI obteve o menor speedup (2.20x) no cenário de 30 iterações.

- **Causa:** A operação MPI_Allreduce traz a cópia de memória inter-processos a cada iteração. Enquanto threads compartilham dados instantaneamente via RAM, processos MPI precisam serializar e transmitir dados.
- **Escalabilidade:** Apesar de mais lento em uma única máquina, o MPI é a única solução que permite escalar horizontalmente. O overhead aqui é o preço pago pela capacidade de rodar em cluster.

3.2.3. Carga Curta e Inicialização

No gráfico de "Speedup Comparativo", observamos o comportamento com apenas 1 iteração:

- **Pthreads (2.86x)** superou OpenMP (2.60x) neste cenário. Isso mostra que o pool de threads do OpenMP pode ter um custo de inicialização maior que a criação crua de threads.
- **MPI (1.96x)** sofreu uma queda expressiva de eficiência. Isso demonstra que o custo fixo de inicializar o ambiente MPI (MPI_Init, configuração de comunicadores) e distribuir os dados iniciais domina o tempo de execução quando a carga de trabalho é pequena.

4. Conclusão

A análise indica que para ambientes multi core de memória compartilhada, **OpenMP** é a escolha melhor devido à simplicidade de implementação e alto desempenho, abstraindo complexidades de sincronização.

A implementação **MPI**, apesar de ter apresentado menor desempenho geral nos testes locais por causa do *overhead* de comunicação e sincronização global (Allreduce), estava correta e robusta. Ela é a principal para cenários de Big Data onde o dataset M supera a memória RAM de um único computador, o que faz com que o custo de comunicação pela capacidade de processamento distribuído seja justificado.

Todas as versões produziram resultados iguais (Checksum 5002634), ou seja, as paralisações foram implementadas de maneira correta.